Micro-exercise 1 — Types, Overflow, and Floating Point

Weight: 10% of your final grade, shared jointly with Micro-exercise 2 in week 3. Scored 0–100%, not pass/fail.

> **CLOSED-BOOK, LIKE THE EXAM** No internet, no AI assistance (Cline stays untouched — same AI-off policy as Lab 0), no personal notes, no textbook, no help from a neighbor. Everything you need was in this week's two lectures. This is the first graded exercise held to exam rules — get used to it now.

How you'll be checked

There is no starter-repo project for this exercise — write one small .cpp file (all three tasks can live in one file, in order) and compile it directly:

```
g++ -Wall -Wextra -std=c++14 exercise.cpp -o exercise
./exercise
```

Near the end of the lab, a TA will come to your machine, run your program, and check it against the three tasks below. No screenshots, no git submission — your individual Gitea repo doesn't exist until week 4.

1. Print type sizes

Print sizeof for at least these six types, each on its own clearly labeled line: bool, char, int, long, long long, double.

```
std::cout << "bool: " << sizeof(bool) << "\n";
// ... one line per type
```

2. Trigger an overflow — on purpose

Write code that deliberately overflows an unsigned type — decrement past zero, or increment past its maximum — and print the resulting value. Then, as a comment right below it, write one sentence explaining WHY the value came out the way it did. “It's garbage” is not an acceptable explanation; “it wrapped because it ran out of room” is closer.

```
unsigned int x = 0;
--x;
std::cout << x;
// explanation: ...
```

3. Compare two doubles — wrong, then right

First, show the wrong way: compare 0.1 + 0.2 to 0.3 with == and print whether they're “equal”. Then fix it: compare them again using std::abs (from <cmath>) and a small tolerance, and print that they're “close enough.”

```
#include <cmath>

double a = 0.1 + 0.2, b = 0.3;
// wrong way: if (a == b) ...
// right way: if (std::abs(a - b) < /* your tolerance */) ...
```

Check you are ready

Before the TA reaches your machine, confirm: all three tasks compile with zero warnings under -Wall -

Wextra; Task 2's explanation actually says WHY the value changed, not just what printed; Task 3 shows both the wrong comparison and the fixed one, not just the fix.

Additional Exercises (Take-Home, Submit With This Lab)

Added 2026-09-27: 4 short programming questions, on top of the three tasks above, not instead of them. Do these outside the 2-hour session and submit them alongside it. None of the four need if/while/for — those aren't taught until next week; each is solvable with the arithmetic and I/O you already know.

> **HOW YOU'LL BE CHECKED** No online judge. For each exercise, your submission is one source file plus a screenshot of every test case in that exercise's table below, showing your program compiling and running with that exact input and producing output that matches (all screenshots go into one combined report PDF — see Submission below for exact file names). Test your own code against every listed test case BEFORE you submit — a screenshot that doesn't match the expected output will not be accepted as correct.
>
> **SUBMISSION** Push to your git repository, under a `lab-output/W02/` directory (create it if it isn't there yet): `Ex01.cpp`, `Ex02.cpp`, `Ex03.cpp`, `Ex04.cpp` — one source file per exercise, named exactly like this (capital E, two digits). `W02-homework.pdf` — a single PDF containing a screenshot of every test case's run, organized by exercise in order. Then `git add lab-output/W02`, `git commit -m "W02 additional exercises"`, `git push` — the same three commands used for every submission in this course (see `docs/GIT_BASICS.md` in the starter repo if you haven't used them yet; formally taught in week 4, but they work now). Default deadline, unless your TA announces otherwise: before Week 3's lab begins.

Output format note: exact wording/labels in your output are up to you (e.g. printing just “50” or “Result: 50” are both fine) — what's checked is that the correct value is clearly visible. For Ex02, don't worry about the exact number of decimal places shown — formatting output precisely is next week's material; just make sure the VALUE is right.

Ex1 — Ten Times the Input

Read an integer x (x < 10000) from standard input. Output x multiplied by 10. You may assume the input is always a valid integer less than 10000 — no need to validate it.

Input	Expected output
5	50
0	0
-3	-30
9999	99990
-9999	-99990

Ex2 — Area of a Circle

Read a floating-point number as the radius of a circle from standard input, and output the circle's area (area = π × radius × radius). Define π as a named const double rather than typing the literal inline more than once. You may assume the input never causes overflow or underflow.

Input	Expected output
1.0	3.14159 (≈ π)
2.5	19.635
0	0
10	314.159
0.5	0.785398

Ex3 — Seconds to hh:mm:ss

Read a total number of seconds (0 ≤ seconds ≤ 10000) from standard input, and convert it to hours:minutes:seconds form, output as hh:mm:ss (or any clearly labeled equivalent — zero-padding to

two digits is not required, since output formatting with setw/setfill isn't taught until week 3). You may assume the input always satisfies 0–10000.

Input	Expected output
0	0:0:0
59	0:0:59
60	0:1:0
3661	1:1:1
10000	2:46:40

Ex4 — Split a Three-Digit Number

Read a three-digit integer (100–999) from standard input, and output its three digits (hundreds, tens, ones) separately. Any output order is acceptable. You may assume the input is always a valid three-digit integer.

Input	Expected output
123	1 2 3
999	9 9 9
100	1 0 0
405	4 0 5
807	8 0 7